

00 - Mobile Feature-Driven Development Overview

1. Purpose

This document is the operating manual for building the Android-first mobile app for Financial Dream Planner. The goal is not to produce a giant React Native application quickly. The goal is to produce a set of small, replaceable, testable feature slices that can be understood by a human or an AI coding agent without loading the entire repository.

2. Product role

The mobile app is the primary daily-use product. It should answer three questions quickly:

1. Where do I stand financially?
2. What changed?
3. What is the best next action?

Primary tabs:

- Home
- Transactions
- Goals
- Plan
- AI

Secondary destinations such as Accounts, Loans, Notifications, Profile, Settings, Transaction Detail, Goal Detail, and Scenario Detail are pushed screens, not permanent tabs.

3. Repository placement

Add the mobile app at the repository root:

```
financial_dream_planner/  
|-- backend/  
|-- frontend/  
|-- mobile/  
|-- sdk/  
|-- docs/  
|-- postman/  
`-- docker-compose.yml
```

`mobile/` is independently packaged and should have its own `package.json`, lockfile, Expo config, TypeScript config, tests, and EAS configuration.

The top-level `sdk/` remains the canonical typed REST client generated from backend OpenAPI. Prefer a local package dependency during development. If EAS packaging of a parent local dependency becomes awkward, publish the SDK as a private/package-registry artifact rather than copying API contracts manually.

4. Locked mobile stack

```
Expo + React Native + TypeScript  
Expo Router  
TanStack Query  
Zustand for small UI/local state only  
Expo SQLite  
Expo SecureStore  
React Hook Form + Zod  
React Native StyleSheet  
React Native Reanimated  
React Native Gesture Handler  
Shopify FlashList  
Expo Notifications  
Victory Native for standard charts
```

React Native Skia only for custom/premium visualizations
Lucide React Native

Custom native code is limited to a local Expo Kotlin module for Android SMS access.

5. Feature-first folder architecture

```
mobile/
|-- app/                                # Expo Router: thin route files only
|-- src/
|   |-- features/
|   |   |-- auth/
|   |   |-- onboarding/
|   |   |-- home/
|   |   |-- accounts/
|   |   |-- transactions/
|   |   |-- sms-sync/
|   |   |-- goals/
|   |   |-- financial-health/
|   |   |-- plan/
|   |   |-- scenarios/
|   |   |-- loans/
|   |   |-- ai/
|   |   |-- notifications/
|   |   |-- settings/
|   |-- components/
|   |   |-- ui/
|   |   |-- layout/
|   |-- api/
|   |-- database/
|   |-- design/
|   |-- providers/
|   |-- hooks/
|   |-- lib/
|   |-- utils/
|   |-- constants/
`-- modules/
    |-- finance-sms/                    # local Expo native module
```

6. Feature module template

Every non-trivial feature should follow this pattern when useful:

```
features/transactions/
|-- screens/
|   |-- TransactionsScreen/
|       |-- TransactionsScreen.tsx
|       |-- styles.ts
|       |-- index.ts
|-- components/
|   |-- TransactionList/
|   |-- TransactionRow/
|   |-- TransactionFilters/
|   |-- SyncStatus/
|-- hooks/
|   |-- useTransactions.ts
|   |-- useTransaction.ts
|   |-- useTransactionFilters.ts
|-- services/
```

```
|  `-- transactions.api.ts
|-- schemas/
|-- store/
|-- utils/
|-- types.ts
`-- index.ts
```

Do not force every folder into every feature. Create structure based on responsibility, not ceremony.

7. Modularity rules

- Route files compose feature screens; they do not contain business logic.
- Screens compose components; they should not directly call low-level API methods.
- API access lives behind feature services and TanStack Query hooks.
- Use SDK DTOs when they already describe API data. Create view-model types only when the UI transforms data.
- Local feature state stays inside the feature.
- Cross-feature imports should use a small public boundary, never deep-import internals.
- Global `components`, `hooks`, `utils`, and `types` are for truly cross-feature code only.
- Prefer roughly 100-300 lines for normal files. Larger is acceptable when the responsibility is still singular and clear.
- Split by responsibility, not arbitrary line count.
- Do not create giant barrel files that pull half the app into one import graph.

8. State ownership

TanStack Query

Owns server state:

- user profile
- transactions
- accounts
- goals
- plan
- scenarios
- loans
- notifications
- AI job metadata

Zustand

Owns small transient client state:

- currently selected local filters
- temporary UI preferences
- sheet/modal state when routing is not appropriate
- uncommitted UI-only selections

Do not mirror TanStack Query server data into Zustand.

SQLite

Owns local/offline operational data:

- pending transaction outbox
- locally parsed SMS transaction candidates
- sync cursors/state
- recent cached transaction records where useful
- merchant mapping cache
- parser version metadata

SecureStore

Owns secrets only:

- refresh token
- device-local auth secret material if later required

Access token stays in memory.

9. API flow

Feature component

v

TanStack Query hook

v

Feature API wrapper

v

@financial-dream-planner/sdk

v

Express REST API

No random Axios calls, raw fetch calls, and SDK calls scattered across screens.

Central API transport handles:

- base URL
- access token
- refresh flow
- single-flight refresh
- one retry after 401
- normalized API errors
- request cancellation

10. Offline model

The app is selectively offline-capable, not fully offline-first.

Works offline:

- recently cached Home snapshot
- recent transactions
- SMS scan and local parse
- manual transaction creation into outbox
- selected lightweight local edits

Requires internet:

- AI chat
- external research
- fresh financial data
- plan regeneration
- server-side scenario calculations

Backend remains authoritative.

11. UI system

MVP is light mode only, but tokens must be theme-ready.

Semantic token groups:

`color.background`

`color.surface`

`color.surfaceRaised`

```
color.textPrimary
color.textSecondary
color.border
color.primary
color.success
color.warning
color.danger
color.info
spacing.*
radius.*
typography.*
elevation.*
motion.*
```

Mobile and web share concepts, not component code.

12. Mobile design rules

- Use platform-appropriate navigation; do not port desktop sidebar patterns to mobile.
- Default page horizontal padding: approximately 16dp.
- Minimum interactive target: 48dp.
- Use tokenized typography.
- Use skeletons for unknown initial state.
- Use cached content immediately when available.
- Use a spinner only for short scoped actions such as button submission.
- Use bottom sheets for mobile filters and lightweight contextual selection.
- Respect reduced motion.
- Never communicate risk/positive/negative only through color.
- Charts require textual summaries for accessibility.

13. Feature completion contract

A feature is not complete when its happy-path screen renders. It is complete when it has:

1. route(s)
2. screen/component structure
3. API integration
4. loading state
5. empty state
6. recoverable error state
7. offline behavior where applicable
8. accessibility labels/roles
9. tests for important logic
10. acceptance criteria satisfied
11. no sensitive data logged
12. documentation updated

14. Feature dependency graph

```
Foundation/App Shell
  v
Auth
  v
Accounts + Financial State
  v
Transactions + SMS Sync
  v
Onboarding completion / Initial snapshot
  v
```

Goals + Financial Health

v

Plan + Projection + Roadmap

v

Loans + Scenarios + Drift

v

AI + Research

v

Notifications + Settings + Release hardening

Some features can overlap in implementation, but this dependency order should drive integration.

15. Global Definition of Done

A production-ready feature must:

- compile with strict TypeScript
- pass lint/typecheck
- pass relevant unit/component tests
- render correctly on at least one small Android device and one large Android device/emulator
- handle offline/API failure without blank-screening
- have no known high-severity accessibility issue
- have no token/SMS/raw financial secret logging
- use shared design tokens/components when applicable
- keep route files thin
- keep financial calculations on backend/deterministic engine, not in AI or duplicated in mobile